Exercise 2: E-commerce Platform Search Function

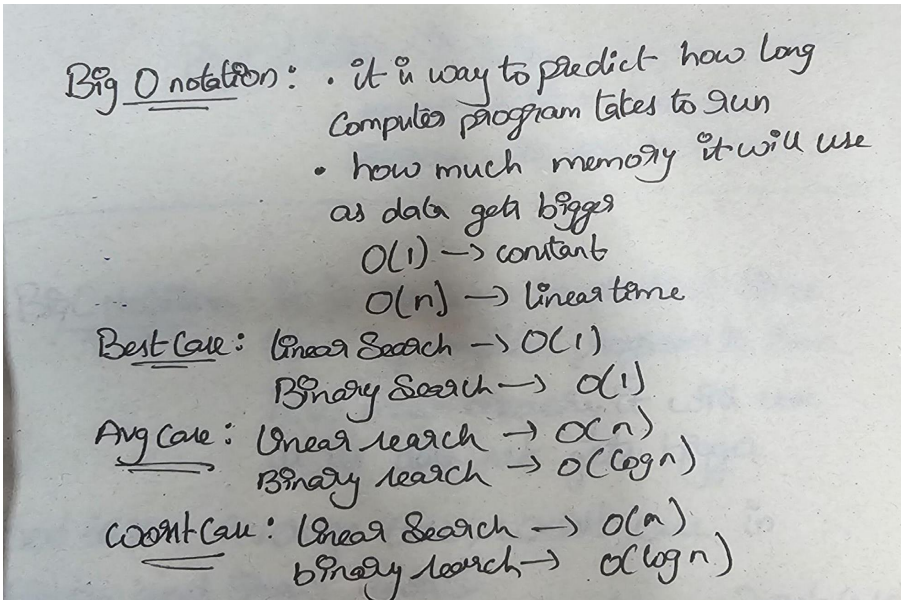
Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. **Understand Asymptotic Notation:**
    - Explain Big O notation and how it helps in analyzing algorithms.
    - Describe the best, average, and worst-case scenarios for search operations.
2. **Setup:**
    - Create a class **Product** with attributes for searching, such as **productId, productName**, and **category**.
3. **Implementation:**
    - Implement linear search and binary search algorithms.
    - Store products in an array for linear search and a sorted array for binary search.
4. **Analysis:**
    - Compare the time complexity of linear and binary search algorithms.
    - Discuss which algorithm is more suitable for your platform and why.

Solution:



```java
import java.util.*;
class Item {
    int itemCode;
    String itemTitle;
    String itemType;
    Item(int itemCode, String itemTitle, String itemType) {
        this.itemCode = itemCode;
        this.itemTitle = itemTitle;
        this.itemType = itemType;
    }
    void showItem() {
        System.out.println("Item Code : " + itemCode);
        System.out.println("Item Name : " + itemTitle);
        System.out.println("Item Category : " + itemType);
    }
}
public class ProductFinder {
    static Item searchLinearly(Item[] itemList, int wantedCode) {
        for (Item currentItem : itemList) {
            if (currentItem.itemCode == wantedCode) {
                return currentItem;
            }
        }
        return null;
    }
    static Item searchBinary(Item[] itemList, int wantedCode) {
        int startIndex = 0;
        int endIndex = itemList.length - 1;
        while (startIndex <= endIndex) {
            int middleIndex = startIndex + (endIndex - startIndex) / 2;
            if (itemList[middleIndex].itemCode == wantedCode) {
                return itemList[middleIndex];
            }
            if (itemList[middleIndex].itemCode < wantedCode) {
                startIndex = middleIndex + 1;
            } else {
                endIndex = middleIndex - 1;
            }
        }
        return null;
    }
    public static void main(String[] args) {
        Scanner inputReader = new Scanner(System.in);
        System.out.println("E-Commerce Product Search");
        System.out.print("Enter number of items: ");
        int totalItems = inputReader.nextInt();
        inputReader.nextLine();
        Item[] itemArray = new Item[totalItems];
        for (int counter = 0; counter < totalItems; counter++) {
            System.out.println("\nItem " + (counter + 1));
            System.out.print("Enter Item Code: ");
            int codeInput = inputReader.nextInt();
            inputReader.nextLine();
            System.out.print("Enter Item Name: ");
            String nameInput = inputReader.nextLine();
            System.out.print("Enter Item Category: ");
            String categoryInput = inputReader.nextLine();
            itemArray[counter] = new Item(codeInput, nameInput, categoryInput);
        }
        System.out.print("\nEnter Item Code to Search: ");
        int codeToFind = inputReader.nextInt();
        Item resultLinear = searchLinearly(itemArray, codeToFind);
        System.out.println("\nLinear Search Result:");
        if (resultLinear != null) {
            resultLinear.showItem();
        } else {
            System.out.println("Item not found.");
        }
        Item[] sortedItemArray = Arrays.copyOf(itemArray, itemArray.length);
        Arrays.sort(sortedItemArray, Comparator.comparingInt(obj -> obj.itemCode));
```

```
        Item resultBinary = searchBinary(sortedItemArray, codeToFind);
        System.out.println("\nBinary Search Result:");
        if (resultBinary != null) {
            resultBinary.showItem();
        } else {
            System.out.println("Item not found.");
        }
        inputReader.close();
    }
}
```

Output:

```
Enter Item Code to Search: 201

Linear Search Result:
Item Code : 201
Item Name : lenovo thinkpad
Item Category : laptop

Binary Search Result:
Item Code : 201
Item Name : lenovo thinkpad
Item Category : laptop

C:\Users\rehaa\Cognizant>
```

Suitability:

Binary Search is more Suitable since there are more than 1000 documents containing the Products details so Binary Search works Faster. Seeing the Complexity difference the Linear Search works O(n) where as Binary Search works O(log n). So its better to use Binary Search only limitation that we got is that the Elements of the Search to be in Sorted order so that the Search happens Efficiently…

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. **Understand Recursive Algorithms:**
    - Explain the concept of recursion and how it can simplify certain problems.
2. **Setup:**
    - Create a method to calculate the future value using a recursive approach.
3. **Implementation:**
    - Implement a recursive algorithm to predict future values based on past growth rates.
4. **Analysis:**
    - Discuss the time complexity of your recursive algorithm.
    - Explain how to optimize the recursive solution to avoid excessive computation.

Solution:

Recursion:

It is a programming technique in which a method calls itself to solve a problem. We break a large problem into smaller sub problems until all conditions are satisfied. Recursion can simplify problems that have repetitive or hierarchical structures.

```java
import java.util.Scanner;
public class WealthEstimator {
    static double estimateFutureWorth(double baseValue, double rateOfGrowth, int durationYears) {
        if (durationYears == 0) {
            return baseValue;
        }
        return estimateFutureWorth(
                baseValue * (1 + rateOfGrowth / 100),
                rateOfGrowth,
                durationYears - 1
        );
    }
    public static void main(String[] args) {
        Scanner readerObj = new Scanner(System.in);

        System.out.println("Financial Forecast Tool");

        System.out.print("Enter current amount: ");
        double initialValue = readerObj.nextDouble();

        System.out.print("Enter annual growth rate (%): ");
        double growthPercent = readerObj.nextDouble();

        System.out.print("Enter number of years: ");
        int totalDuration = readerObj.nextInt();

        double finalPrediction = estimateFutureWorth(initialValue, growthPercent, totalDuration);
        System.out.printf("\nPredicted value after %d years: %.2f\n", totalDuration, finalPrediction);
        readerObj.close();
    }
}
```

Output:

```
C:\Users\rehaa\Cognizant>javac WealthEstimator.java

C:\Users\rehaa\Cognizant>java WealthEstimator
Financial Forecast Tool
Enter current amount: 980000
Enter annual growth rate (%): 1.75
Enter number of years: 4

Predicted value after 4 years: 1050421.85
```

Complexity:

For every year : T(n) = T(n-1) + O(1)
Time and Space Complexity will be O(n). Where as n is number of years.

Optimization:

The recursive solution can be optimized by: Iterative approach or by Direct formula

This eliminates repeated recursive calls and improves efficiency for large values of years.